

mapping_pathfinding

Kartierung, Lokalisierung und Pfadplanung für die Duckiebot-City-Challenge

Nico Sander

21. Juli 2026

1 Systemüberblick

mapping_pathfinding ist ein ROS-1-Paket (Noetic), mit dem ein Duckiebot eine kleine Stadt aus Straßen und Kreuzungen selbstständig erkundet, eine Karte davon aufbaut und anschließend eine angesagte Folge von Toren in der schnellsten gefundenen Reihenfolge abfährt.

Die Mission besteht aus zwei Phasen:

MAPPING (ungestoppt) Der Bot fährt jede Straße der Stadt mindestens einmal ab. Dabei erfasst er, welches *Tor* (ein AprilTag über einer Straße) zu welcher Straße gehört, und misst, wie lange er für jede Straße und jedes Abbiegemanöver tatsächlich braucht.

GATE_RUN (gestoppt) Die Torreihenfolge wird vor Ort als Liste von Tag-IDs angesagt. Der Bot wird an einer bekannten Straße abgesetzt, plant anhand der Karte und der gemessenen Zeiten aus der Mapping-Phase eine Route durch die Tore in dieser Reihenfolge und fährt sie ab.

DONE Nach dem letzten Tor hält der Bot an und meldet die benötigte Zeit.

Die Stadt wird als Graph modelliert: Knoten sind Kreuzungen, Kanten sind Straßen. Jede Kreuzung hat bis zu vier nummerierte *Ports* (1–4), die zyklisch angeordnet sind – für einen Bot, der über Port p einfährt, liegt Port $p + 1$ rechts und $p - 1$ links. Da Wendemanöver verboten sind, wird nicht über Knoten geplant, sondern über gerichtete Zustände (**Knoten, Einfahrt-Port**) – “ich nähere mich dieser Kreuzung und bin über diesen Port eingefahren”. Dieser Zustand identifiziert zugleich die gerade befahrene Straße, was es dem Planer erlaubt, einzelne Straßen und Tore gezielt anzusteuern. Die Karte selbst liegt in einer einzigen JSON-Datei (`config/city.json`), die alle Nodes lesen – es gibt also immer nur genau eine Definition der Stadt.

Das Paket trennt bewusst zwischen dünnen ROS-Nodes und einem ROS-freien Kern: sämtliche Graph-, Planungs-, Zeitmess- und Filterlogik steckt in reinen Python-Modulen ohne ROS-Import. Dadurch ist der gesamte Entscheidungskern durch Offline-Unittests abgedeckt (derzeit 209), die ohne Roboter laufen.

[PLATZHALTER: ROS-Node-Graph]

Ein Blockdiagramm des laufenden ROS-Graphen. Pro Node ein Kasten (`detect_lane`, `detect_signs`, `detect_intersection`, `switch_control`, `control_wheels`, `mapping_pathplanning`, `visualization`, `dashboard`), dazu Kamera und Radansteuerung als externe Kästen, verbunden durch Pfeile mit dem jeweiligen Topic-Namen. Die wichtigen Kanten sind: Kamera → `detect_lane` und `detect_signs`; `/detect/lane_borders` → `detect_intersection`; `/detect/intersection` und `/detect/sign` → `switch_control`; `/switch/mode` und `/switch/turn_direction` → `control_wheels` und → `mapping_pathplanning`; `/detect/sign_detections` → `mapping_pathplanning`; und die Kante, die den Regelkreis schließt: `/plan/turn_command` → `switch_control`. Ebenfalls einzeichnen: `/mapping/state` → `visualization`. Optisch hervorheben lohnt sich die Schichtung – Wahrnehmung (links), Verhalten (Mitte), Mission/Planung (rechts) – und die Tatsache, dass die Missionsschicht auf die Verhaltensschicht zurückwirkt. Ein Screenshot von `rqt_graph` tut es zur Not auch, eine handgezeichnete Version ist aber deutlich lesbarer, weil sie die Debug-Topics weglassen kann.

2 Beschreibung der Kernkomponenten

2.1 Wahrnehmung

detect_lane.py Segmentiert das Kamerabild mit einem PyTorch-U-Net in weiße Fahrbahnmarkierungen, gelbe Mittellinie und rote Haltelinien. Daraus berechnet der Node den Querabstandsfehler – wie weit der Bot aus der Spurmitte steht – und veröffentlicht ihn auf `/detect/lane`. Zusätzlich publiziert er die geometrischen Spurgrenzen (inklusive Position einer roten Linie im Bild) auf `/detect/lane_borders` sowie Debug-Bilder für das Dashboard. Die gesamte Wahrnehmungspipeline läuft mit 30 Hz auf dem Laptop, etwa 20 ms pro Bild bei einem Budget von 33 ms.

detect_signs.py Erkennt AprilTags mit `pupil_apriltags` und publiziert auf zwei Topics, weil die beiden Abnehmer Unterschiedliches brauchen. `/detect/sign` enthält nur die ID des nächstgelegenen Tags und ist ungefiltert – `switch_control` muss Kreuzungsschilder in jeder Größe sehen. `/detect/sign_detections` enthält jede Detektion mit Eckpunkten, Fläche in Pixeln und einer Annahme-/Verwerfen-Entscheidung; der Missions-Node braucht die Fläche, um zu unterscheiden, ob ein Tor auf der aktuellen Straße hängt oder erst hinter der Kreuzung.

detect_intersection.py Ein kleiner Logik-Node, der aus der Spurgeometrie einen Kreuzungszustand ableitet: keine Kreuzung, Kreuzung nähert sich, Haltelinie erreicht. Das geschieht allein über die vertikale Position der erkannten roten Linie, ohne das Bild erneut anzufassen.

2.2 Verhalten und Aktorik

switch_control.py Der Verhaltensautomat. Er entscheidet, *was der Bot gerade tut* – Spur folgen, sich einer Kreuzung nähern, an der Haltelinie stehen oder die Kreuzung queren – und veröffentlicht das als `/switch/mode` zusammen mit der Abbiegerichtung auf `/switch/turn_direction`. Die Richtung kommt im Normalfall vom Missions-Node über `/plan/turn_command`; ist dieses Kommando veraltet oder fehlt es, fällt der Node auf sein älteres Verhalten zurück und wählt die Richtung anhand des Kreuzungsschilds – ein ausgefallener Missions-Node führt so zu degradiertem statt zu gar keinem Fahrverhalten. Kreuzungsquerungen sind geschlossen geregelt: das Manöver endet, sobald wieder Fahrbahnmarkierungen zu sehen sind, nicht wenn ein Timer abläuft – die konfigurierte Dauer ist nur ein Timeout. Genau das macht den Stack tolerant dagegen, dass der Bot nicht exakt rechtwinklig an der Haltelinie steht.

control_wheels.py Der einzige Node, der mit den Motoren spricht. Er übersetzt den aktuellen Fahrmodus in Radkommandos: beim Spurfolgen läuft ein PID-Regler auf dem Querabstandsfehler (mit tiefpassgefiltertem D-Anteil, damit er bei 30 Hz stabil bleibt), während einer Querung fährt er einen festen Bogen für die kommandierte Richtung, bis `switch_control` das Manöver beendet. Gemessene Latenz von Bildaufnahme bis Kommando: rund 75 ms.

2.3 Mission, Kartierung und Planung

mapping_pathplanning.py Der Node, dem die Mission gehört, und die größte Einzelkomponente des Pakets. Er erledigt vier Dinge. (1) *Lokalisierung*: Koppelnavigation auf dem Graphen – die geglaubte Position rückt um eine Straße weiter, sobald `switch_control` eine abgeschlossene Querung meldet. (2) *Kartierung*: er schreibt Tore auf die Straßen, auf denen sie gesehen wurden. (3) *Planung*: nach jeder Querung wird die gesamte Restroute neu geplant – bei dieser Stadtgröße billig, und es bedeutet, dass eine korrigierte Position sofort zu einer korrigierten Route führt. (4) *Kommandieren*: er publiziert das nächste Abbiegekommando auf `/plan/turn_command` und schließt damit den Regelkreis. Zusätzlich veröffentlicht er den vollständigen Missionszustand (Karte, Tore, Pfad, Position, Phase) als JSON auf `/mapping/state` und bietet einen Service `start_gate_run` für den Phasenwechsel an.

planner.py (ROS-frei) Dijkstra über die gerichteten Zustände (Knoten, Einfahrt-Port). Wendemanöver werden nicht durch eine Prüfung verboten, sondern sind strukturell nicht ausdrückbar, weil keine Richtungstabelle einen Port auf sich selbst abbildet. Kanten sind mit einer geschätzten Dauer gewichtet – ein Zug kostet **Halt + Abbiegen + Straße** – damit die Suche wirklich schnellere und nicht bloß kürzere Routen bevorzugt; eine Rechtskurve ist deutlich billiger als eine Linkskurve. In der Mapping-Phase treibt dasselbe Modul eine gierige Erkundung nach der jeweils nächstgelegenen noch nicht befahrenen Straße.

city_map.py (ROS-frei) Der Graph selbst: Port-Geometrie, Laden und Validieren von `city.json` sowie `GraphMap`, der Live-Kartenzustand, der festhält, welche Straßen befahren wurden und welches Tor auf welcher Straße hängt. Kantenschlüssel sind richtungsunabhängig (`A1__B1` von beiden Enden), womit sich “ein Tor gilt für die ganze Straße” von selbst ergibt.

gate_detection.py (ROS-frei) Entscheidet, welchen Tor-Sichtungen geglaubt werden darf. Ein Tor wird dauerhaft auf eine Straße geschrieben – der erste Eintrag gewinnt – weil die Kamera über die Kreuzung hinaus sieht und spätere Sichtungen aus größerer Entfernung und schlechterem Winkel stammen. Drei Filter stehen vor diesem unwiderruflichen Schreibvorgang: im Stand und während einer Querung wird gar nicht kartiert; die Fläche des Tags muss über einem aus aufgezeichneten Fahrten kalibrierten Schwellwert liegen (`gate_min_area`, derzeit 900 px²); und dasselbe Tag muss über mehrere aufeinanderfolgende Bilder auf derselben Straße gesehen werden.

run_timing.py (ROS-frei) Misst, wie lange jede Straße und jedes Abbiegemanöver tatsächlich dauert. Als gemeinsame Zeitbasis dienen die von **switch_control** publizierten Fahrmodus-Übergänge, sodass sich die gemessenen Teile lückenlos und überlappungsfrei zu einem vollen Zug zusammensetzen. Die Mapping-Phase ist ungestoppt, dort zu messen kostet also nichts; der gestoppte Lauf wird anschließend gegen reale Zahlen statt gegen eine pauschale Schätzung geplant. Mehrfachmessungen einer Straße werden über den Median zusammengefasst.

crossing.py und graph_layout.py (ROS-frei) Zwei kleine Hilfsmodule: das erste entscheidet, wann eine Kreuzungsquerung beendet ist (Markierungen wieder sichtbar, nach einer minimalen Blindzeit), das zweite berechnet die Zeichengeometrie der Karte – mit der getesteten Eigenschaft, dass sich gezeichnete Straßen außerhalb eines Knotens niemals kreuzen dürfen.

2.4 Bedienung und Visualisierung

visualization.py Ein Live-Fenster, das die drei von der Challenge geforderten Dinge sichtbar macht: die aufgebaute Karte (Straßen mit den darauf gefundenen Toren), den geplanten Pfad (gestrichelt, mit Schrittnummern) und die geglaubte Position des Bots (aktuelle Straße hervorgehoben, Ring an der nächsten Kreuzung).

dashboard.py Ein Debug-Fenster für die Wahrnehmung, das Rohbild, Segmentierungsmasken und die AprilTag-Rahmen samt gemessener Fläche in einer einzigen Ansicht kachelt.

mission_tui.py / mission_setup.py Ein menügeführtes Frontend für die gesamte Session. Der strukturell wichtige Punkt: *ein roslaunch* bleibt über beide Phasen hinweg am Leben – Karte, Tore und gemessene Zeiten liegen im Missions-Node, ein Neustart zwischen den Phasen würde sie verwerfen. Die TUI besitzt daher den Launch-Prozess und wechselt die Phase über Parameter und den Service-Aufruf. **mission_setup.py** validiert die eingegebenen Antworten (vor allem die Startposition, die vor dem Start gegen den Stadtgraphen geprüft wird) und baut das Launch-Kommando zusammen.

3 Zustands- und Ablaufdiagramme

[PLATZHALTER: Zustandsdiagramm der Missionsphasen]

Der übergeordnete Missionsautomat von **mapping_pathplanning.py** mit drei Zuständen: **MAPPING**, **GATE_RUN** und **DONE**. Einzuzeichnende Übergänge: **MAPPING** → **GATE_RUN**, beschriftet mit “alle Straßen befahren *und* alle Tore gesehen → Halt an der Haltelinie, warten auf den Service **start_gate_run**”; **GATE_RUN** → **DONE**, beschriftet mit “letztes Tor der angesagten Reihenfolge passiert”. Deutlich markieren, dass zwischen **MAPPING** und **GATE_RUN** ein *harter Stopp* liegt, während dessen der Bot physisch umgesetzt wird, und dass der Übergang aus **MAPPING** verlangt, dass die Tore tatsächlich *gesehen* wurden – nicht nur, dass die Straßen befahren wurden.

[PLATZHALTER: Verhaltensautomat (switch_control)]

Der Fahrmodus-Automat: **LANE_FOLLOWING** → **APPROACHING** → **STOPPED** → **CROSSING** → zurück zu **LANE_FOLLOWING**. Jeden Übergang mit seinem Auslöser beschriften: rote Linie oberhalb des Schwellwerts erkannt; Haltelinie erreicht; Haltedauer abgelaufen *und* Abbiegekommando vorhanden; und – der wichtigste – die Querung endet, sobald *wieder Fahrbahnmarkierungen sichtbar* sind, wobei die konfigurierte Abbiegedauer nur als Timeout wirkt. Zusätzlich das **HALT**-Kommando als Übergang in einen Wartezustand einzeichnen sowie **/plan/resume_driving** als jene Kante, die einen umgesetzten Bot ohne Querungsmanöver zurück in **LANE_FOLLOWING** bringt. Es lohnt sich, im selben Diagramm zu annotieren, welches Zeitintervall **run_timing.py** als “Abbiegezeit” und welches als “Straßenzeit” misst.

[PLATZHALTER: Ablaufdiagramm der Tor-Sichtungsentscheidung]

Ein Flussdiagramm für eine einzelne eingehende Detektion aus **/detect/sign_detections**, das entweder in “Tor auf Straße schreiben” oder in “verwerfen” mündet. Entscheidungsrauten der Reihe nach: Führt der Bot überhaupt (nicht gestoppt, nicht querend)? Ist die Tag-ID überhaupt ein Tor? Ist ihre Fläche $\geq$ **gate_min_area**? Wurde dasselbe Paar (Straße, Tor) über **gate_confirm_frames** aufeinanderfolgende Detektionen gesehen? Trägt diese Straße bereits ein Tor (falls ja: verwerfen – der erste Eintrag ist endgültig)? Dieses Diagramm ist die beste Stelle, um zu zeigen, *warum* es jeden Filter gibt; eine einzeilige Anmerkung pro Raute genügt.

[PLATZHALTER: Regelkreis / Sequenz an einer Kreuzung]

Ein Sequenzdiagramm über eine Kreuzung hinweg, mit den Spuren Kamera → `detect_*` → `switch_control` → `control_wheels` und `mapping_pathplanning` als paralleler Spur. Der Ablauf: rote Linie erkannt, Halt, Missions-Node plant neu und publiziert das Abbiegekommando, `switch_control` geht mit dieser Richtung in CROSSING, `control_wheels` fährt den Bogen, Fahrbahnmarkierungen tauchen wieder auf, `switch_control` meldet die abgeschlossene Querung, der Missions-Node rückt seine geglaubte Position um eine Straße weiter und plant neu. Dieses Diagramm macht den geschlossenen Regelkreis lesbar – und die 30-Hz-Innenschleife innerhalb der äußeren Schleife pro Kreuzung.

4 Konfiguration und Tests

Alle abgestimmten Konstanten liegen in `config/config.json` (Zeiten, PID-Parameter, Schwellwerte, Kostenmodell des Planers); die Karte ist `config/city.json`, die rein kosmetische Zuordnung Tor-ID → Farbe steht in `config/gates.json`. Launch-Argumente überschreiben ausgewählte Werte für einen einzelnen Lauf.

Da der Entscheidungskern ROS-frei ist, wird er offline verifiziert: Port-Geometrie und die Invariante “kein Wendemanöver”, Validierung der Stadt, die Filter für Tor-Sichtungen, Erreichbarkeit im Planer aus jedem Startzustand, alle Torreihenfolgen aus allen Startzuständen, gierige Abdeckung, vollständige Missionssimulationen und synthetische Gitterstädte. Zusätzlich betreibt `launch/simulate.launch` den echten Missions-Node gegen einen simulierten Fahrer und testet damit die tatsächliche ROS-Schicht – Topics, Timer, den Phasenwechsel – ohne Kamera, Motoren oder Roboter.